

MERN Stack Internship Assignment Report-Day 11

User Authentication Module using MERN Stack and JWT

Company: INSTA DOT ANALYTICS

Name: Krishnakant Yadav

Roll No: 2023BTCS030

Course: B.Tech III Year (CSIT)

College: Symbiosis University of Applied Sciences

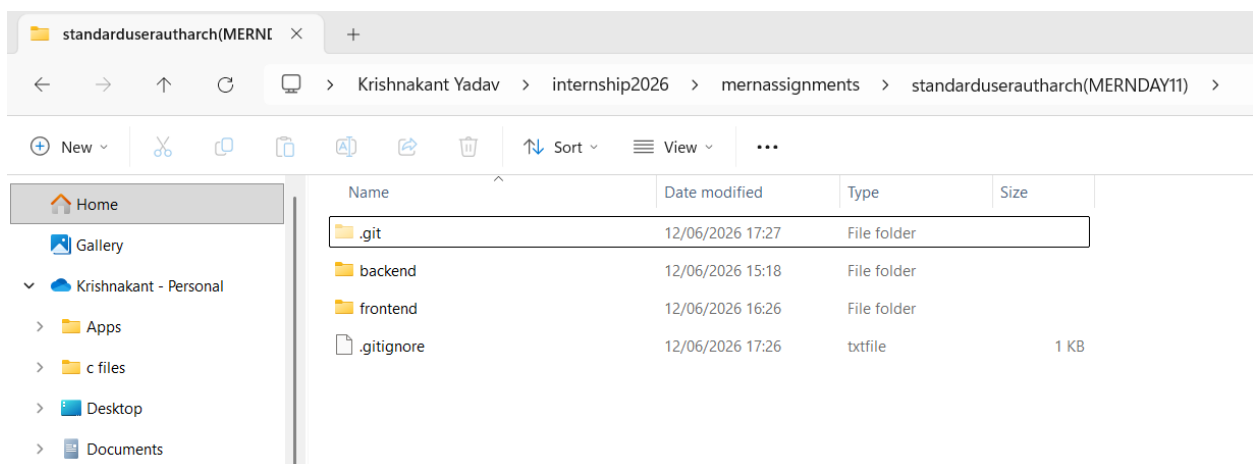
Submission Date: June 12, 2026

1.Objective

The objective of this task was to develop a secure user authentication module using MongoDB, Express.js, React.js and Node.js. The application implements user registration, login functionality, JWT-based authentication and protected routes.

2.Project Structure

The project was divided into frontend and backend modules.



3. Technology Stack Used:

Technology	Usage
React.js	Frontend UI
Vite	React development environment
Node.js	Runtime
Express.js	Backend API
MongoDB Atlas	Database
Mongoose	Database interaction
JWT(jsonwebtoken)	Authentication
Axios	API communication
Git/GitHub	Version control

4. Backend Implementation

4.1 Server Setup:

```
Windows PowerShell x C:\WINDOWS\system32\cmd. x + v
PS C:\Users\Lenovo\internship2026\mernassignments\standarduserautharch(MERNDAY11)\frontend\userauthclient\src> cd ..
PS C:\Users\Lenovo\internship2026\mernassignments\standarduserautharch(MERNDAY11)\frontend\userauthclient> cd ..
PS C:\Users\Lenovo\internship2026\mernassignments\standarduserautharch(MERNDAY11)\frontend> cd .. | cd backend
PS C:\Users\Lenovo\internship2026\mernassignments\standarduserautharch(MERNDAY11)> dir

Directory: C:\Users\Lenovo\internship2026\mernassignments\standarduserautharch(MERNDAY11)

Mode                LastWriteTime         Length Name
----                -
d-----         12/06/2026      15:18         backend
d-----         12/06/2026      16:26         frontend
d-----         12/06/2026      15:45         testingsetups

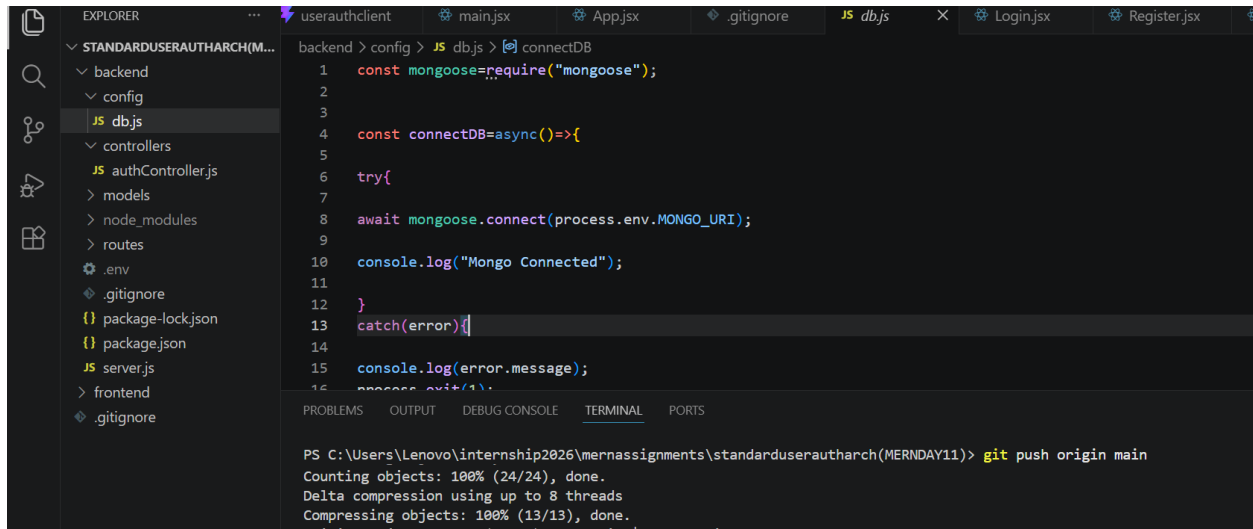
PS C:\Users\Lenovo\internship2026\mernassignments\standarduserautharch(MERNDAY11)> cd backend
PS C:\Users\Lenovo\internship2026\mernassignments\standarduserautharch(MERNDAY11)\backend> npm run dev

> backend@1.0.0 dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter 'rs'
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting 'node server.js'
◇ injected env (3) from .env // tip: % custom filepath { path: '/custom/path/.env' }
Server running
Mongo Connected
|
```

Backend server was successfully started and MongoDB Atlas connection was established.

4.2 Database setup



The screenshot shows a Visual Studio Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'backend', 'config', 'controllers', 'models', 'node_modules', 'routes', and files like '.env', '.gitignore', 'package-lock.json', 'package.json', 'server.js', and 'frontend'. The code editor shows the 'db.js' file with the following code:

```
backend > config > JS db.js > connectDB
1  const mongoose=require("mongoose");
2
3
4  const connectDB=async()=>{
5
6    try{
7
8      await mongoose.connect(process.env.MONGO_URI);
9
10     console.log("Mongo Connected");
11
12    }
13    catch(error){
14
15     console.log(error.message);
16     process.exit(1);
17   }
```

Below the code editor, there is a terminal window showing the output of a git push command:

```
PS C:\Users\Lenovo\internship2026\mernassignments\standarduserautharch(MERNDAY11)> git push origin main
Counting objects: 100% (24/24), done.
Delta compression using up to 8 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (13/13), 2.50 KiB | 0.00 MiB/s | 0:00:01, done.
Total 13 (delta 13), reused 0, compressed 0, 0.00 KiB added.
```

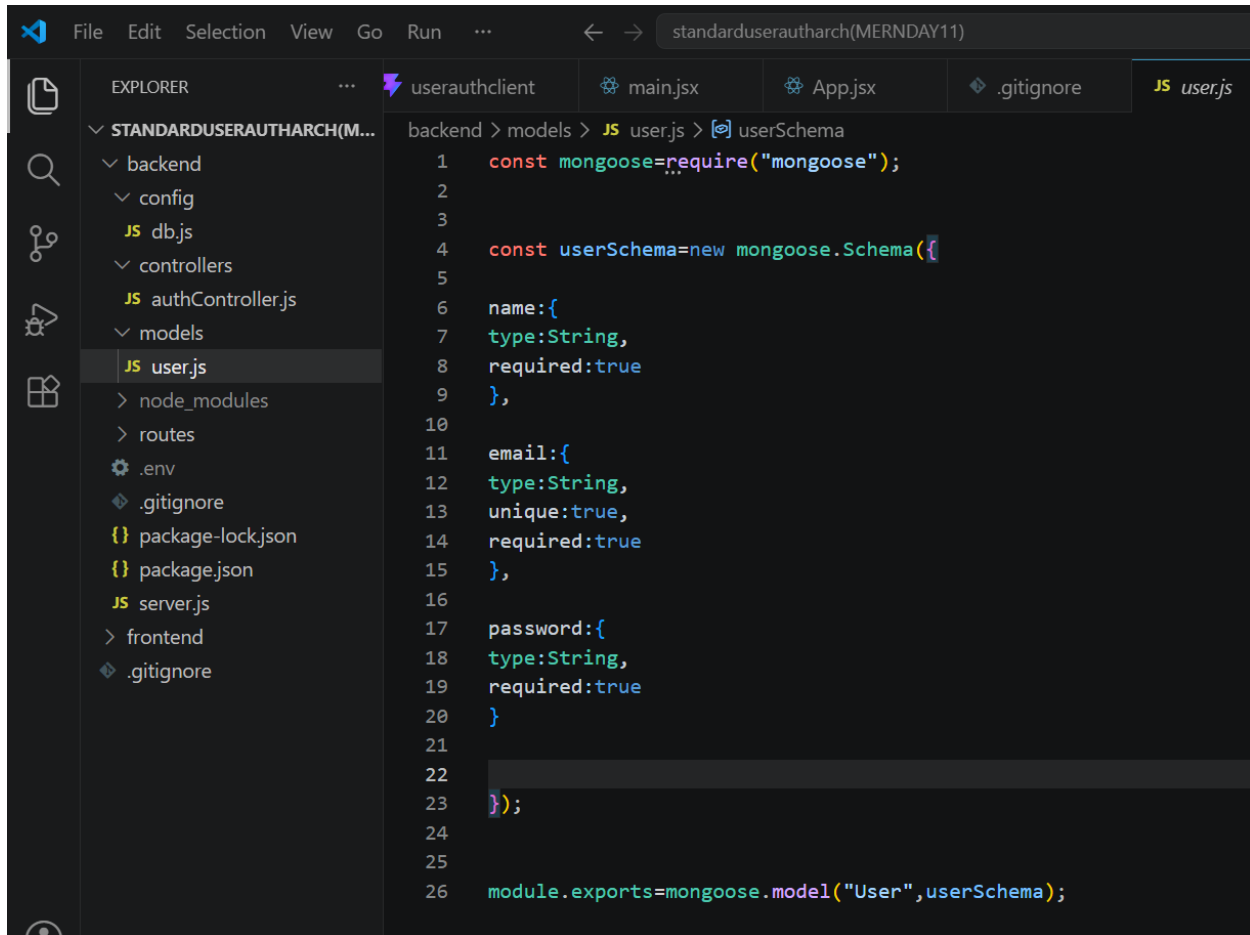
MongoDB Atlas was used as cloud database storage. Database connection was configured using environment variables.

5.3 Environment Variables

Sensitive credentials were stored in environment variables and excluded from Git tracking using .gitignore.

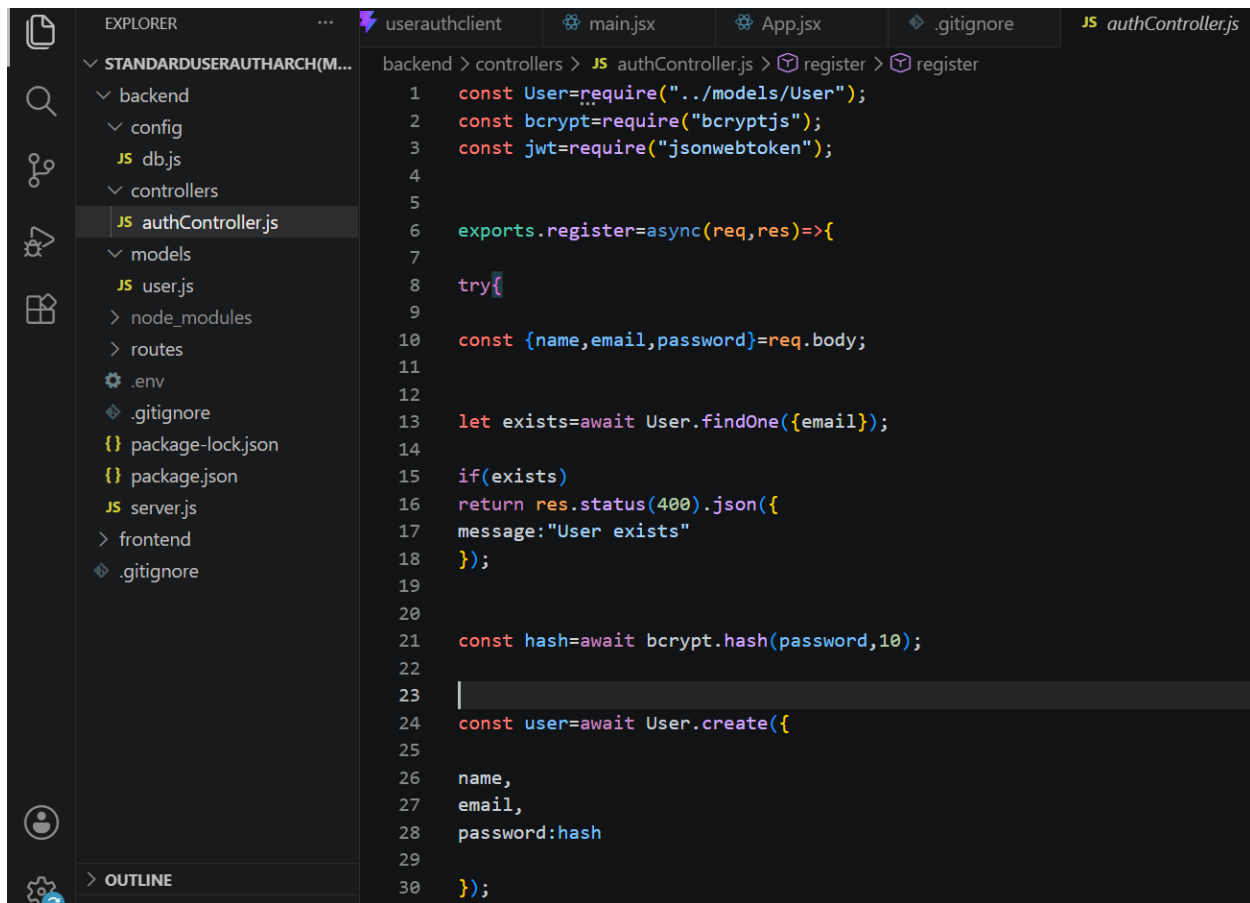
5.4 User Model

User schema was created with fields required for authentication.



```
1  const mongoose=require("mongoose");
2
3
4  const userSchema=new mongoose.Schema({
5
6    name:{
7      type:String,
8      required:true
9    },
10
11    email:{
12      type:String,
13      unique:true,
14      required:true
15    },
16
17    password:{
18      type:String,
19      required:true
20    }
21  });
22
23
24
25
26  module.exports=mongoose.model("User",userSchema);
```

5.6 Login API

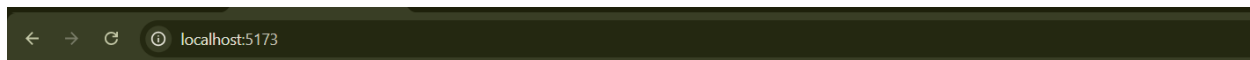


```
1  const User=require("../models/User");
2  const bcrypt=require("bcryptjs");
3  const jwt=require("jsonwebtoken");
4
5
6  exports.register=async(req,res)=>{
7
8    try{
9
10     const {name,email,password}=req.body;
11
12
13     let exists=await User.findOne({email});
14
15     if(exists)
16       return res.status(400).json({
17         message:"User exists"
18       });
19
20
21     const hash=await bcrypt.hash(password,10);
22
23
24     const user=await User.create({
25
26       name,
27       email,
28       password:hash
29
30     });
```

Login API validates credentials and generates a JWT token after successful authentication.

6. Frontend Implementation

6.1 React Setup

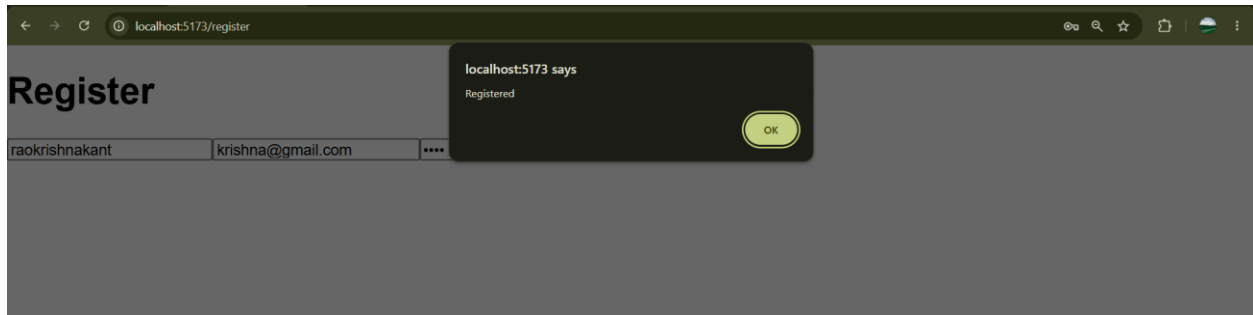


Login

Email	Password	Login
-------	----------	-------

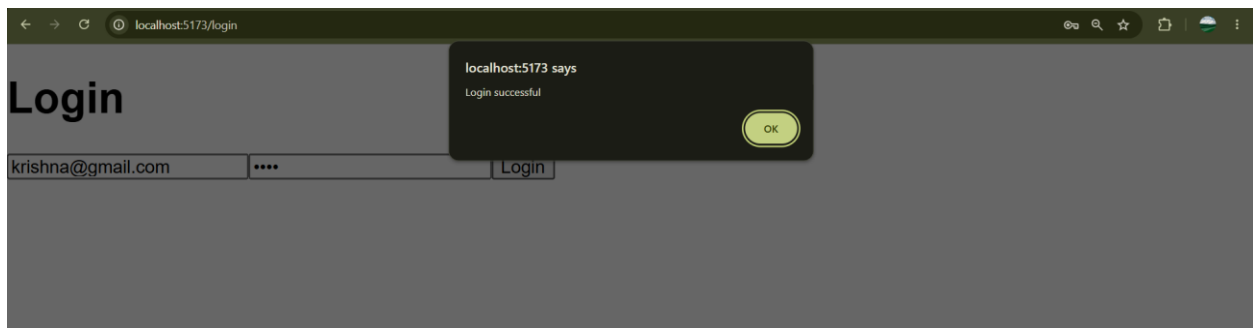
React frontend was running on development server .

6.2 Registration Page



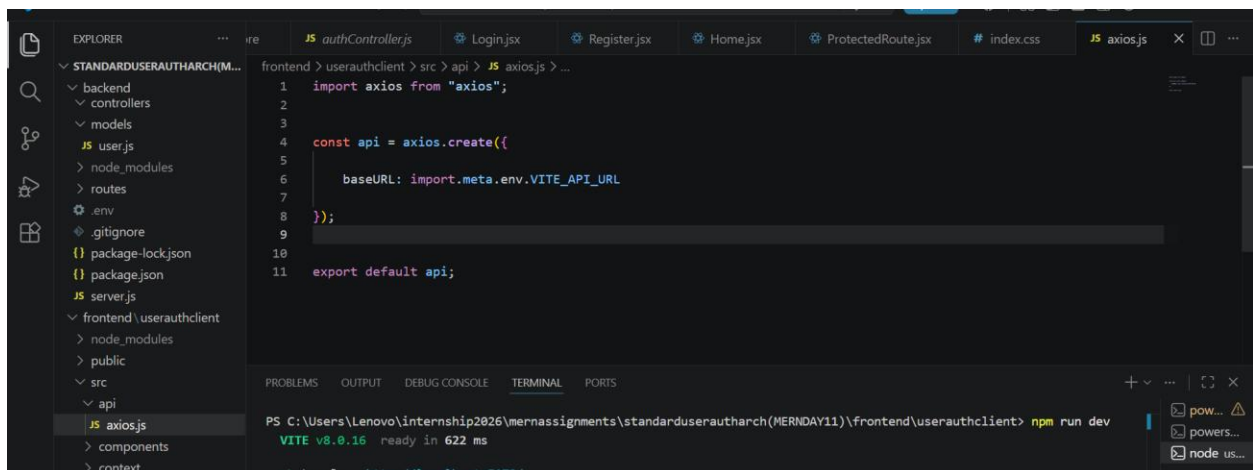
Registration form was created in React and connected with backend API using Axios.

6.3 Login Page



Login functionality was implemented by sending credentials to the Express authentication API.

6.4 API Integration



Axios instance was configured to communicate between React frontend and Express backend.

7. JWT Authentication Flow

User

|

Register

|

Express API

|

MongoDB

|

Login

|

JWT Token

|

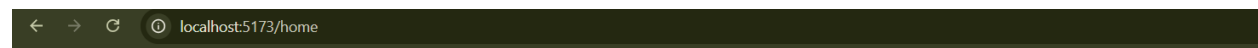
Local Storage

|

Protected Route

JWT token is stored on successful login and used to authorize access to protected pages.

8. Protected Routes



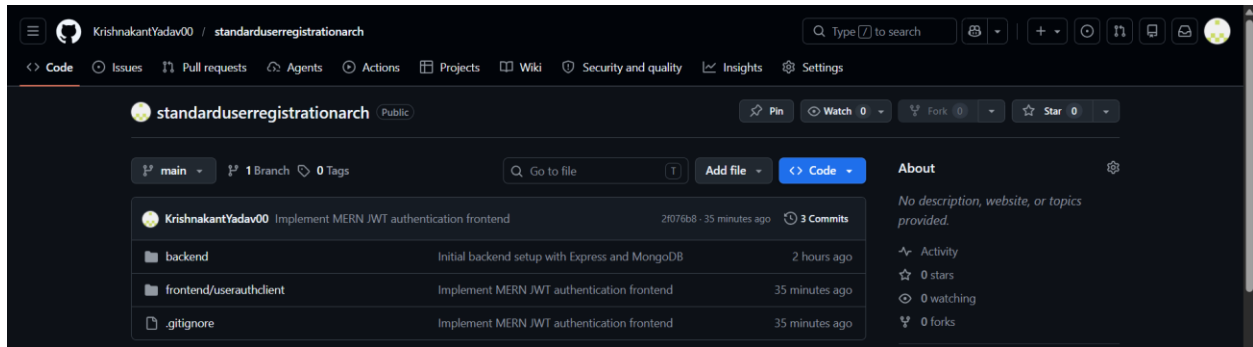
Dashboard

JWT protected area

Logout

ProtectedRoute component checks token availability before allowing access.

9. GitHub Version Control



Source code was committed and pushed to GitHub. Sensitive files were excluded through .gitignore.

10. Testing Results

Test	Result
Backend starts	Passed
MongoDB connection	Passed
User registration	Passed
User login	Passed
JWT generation	Passed
Protected route	Passed

11. Conclusion

The MERN authentication module was successfully implemented. The system provides user registration, login, JWT authentication and route protection.